

# Rilevamento e mitigazione di attacchi DoS in reti SDN

## Introduzione e obiettivi

Il progetto affronta il problema della mitigazione di un attacco Denial of Service in una rete definita da software tramite un emulatore mininet.

L'obiettivo è dimostrare che, tramite un controller SDN, disponendo di una vista globale della rete e della possibilità di agire direttamente sulle tabelle di inoltro dei dispositivi di rete, può isolare il singolo flusso responsabile dell'attacco senza alcun effetto sul traffico legittimo che attraversa gli stessi switch e le stesse porte fisiche.

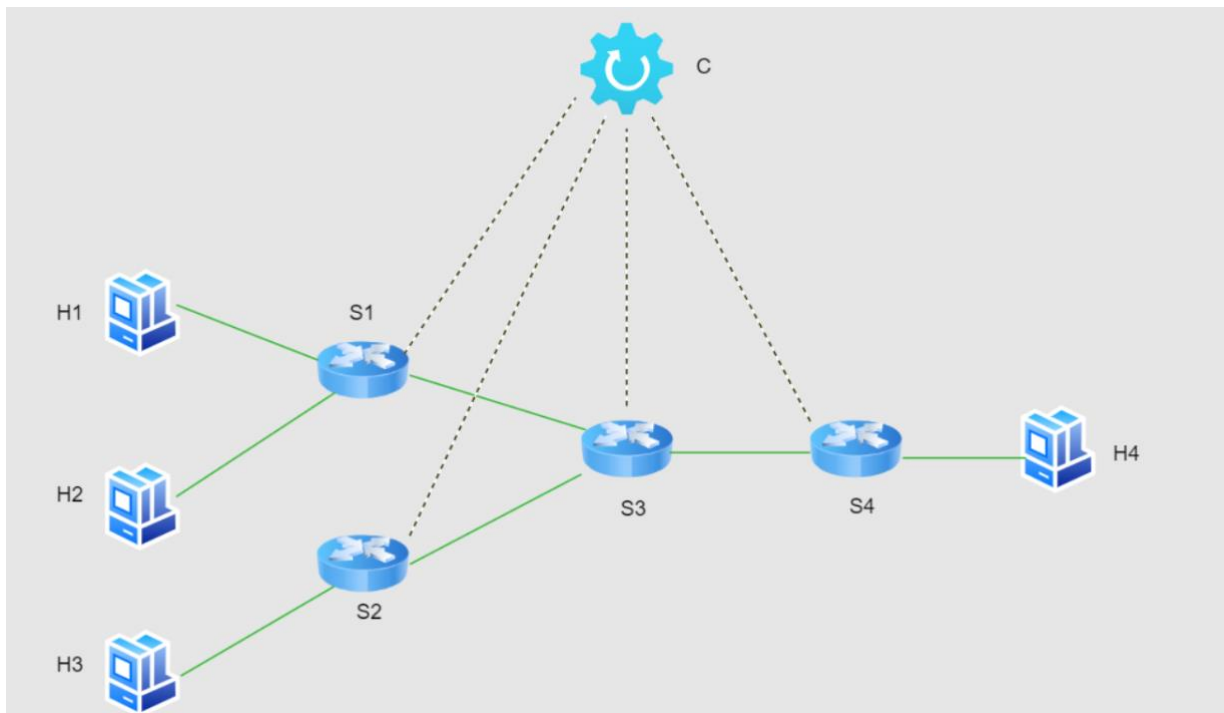
## Tecnologie implementate

| Componente   | Funzionalità                                                            |
|--------------|-------------------------------------------------------------------------|
| Mininet      | Emulazione della topologia di rete                                      |
| Open vSwitch | Switch programmabili che mantengono le flow table e applicano le regole |
| OpenFlow 1.3 | Protocollo di comunicazione tra controller e switch                     |
| Ryu          | Controller centralizzato                                                |
| iperf        | Generazione del traffico                                                |
| Wireshark    | Monitoraggio del traffico sulle interfacce degli switch                 |

## Topologia di rete

La topologia è composta da quattro host e quattro switch e viene realizzata tramite uno script Python. Lo script definisce esplicitamente collegamenti e numerazione delle porte, costruendo una topologia personalizzata.

La rete emula 3 client (H1, H2, H3) che accedono a un servizio ospitato su un server con banda limitata (H4) dove uno degli host è stato compromesso (H2) e genera traffico volumetrico UDP verso il servizio (flood volumetrico) mentre gli altri due client legittimi inviano traffico TCP. Il link verso il server rappresenta un punto di convergenza del traffico e risulta essere un collo di bottiglia per il sistema.



## Collegamenti

Ogni collegamento è configurato con banda e ritardo espliciti. I numeri di porta sono assegnati manualmente rendendo la corrispondenza fra topologia e tabelle di inoltrato del controller deterministica e verificabile.

Gli host sono configurati tramite indirizzi IP e MAC statici che vanno da 10.0.0.1 a 10.0.0.4 senza richiedere l'intervento di un tipo di apprendimento autonomo.

| Collegamento | Porte   | Banda   | Ritardo |
|--------------|---------|---------|---------|
| h1 —> s1     | 0 --> 1 | 10 Mbps | 0.1 ms  |
| h2 —> s1     | 0 --> 2 | 10 Mbps | 0.1 ms  |
| h3 —> s2     | 0 --> 1 | 10 Mbps | 0.1 ms  |
| s1 —> s3     | 3 --> 1 | 20 Mbps | 1 ms    |
| s2 —> s3     | 3 --> 2 | 20 Mbps | 1 ms    |
| s3 —> s4     | 3 --> 1 | 20 Mbps | 1 ms    |
| s4 —> h4     | 2 --> 0 | 10 Mbps | 0.1 ms  |

Infine, le tabelle d'inoltrato di ciascuno switch vengono impartite in modo proattivo all'atto della connessione di ciascun dispositivo in modo da far raggiungere il server da qualsiasi client e viceversa per un totale di 18 regole distribuite sui diversi apparati di rete. Avere una regola per ogni coppia "sorgente-destinazione" all'interno degli switch garantisce che ogni conversazione abbia dei contatori di byte indipendenti, consentendo un monitoraggio "per flusso" e di conseguenza la corretta individuazione della sorgente anomala.

## Rilevamento e mitigazione

Il controller interroga periodicamente tutti gli switch tramite OpenFlow con un intervallo di 3 secondi, legge il contatore di byte di ciascuna rotta e ne ricava il rate come differenza rispetto alla lettura precedente.

### La regola

Un flusso è considerato anomalo quando valgono entrambe le seguenti condizioni:

- È presente contesa, cioè almeno due flussi client-server sono attivi contemporaneamente. In assenza di contesa non vi è alcun servizio che venga negato.
- Il flusso supera la propria quota equa del collo di bottiglia, secondo la relazione:

$$\text{threshold} = \text{FAIR\_FACTOR} * \text{BOTTLENECK\_MBPS} / n\_active$$

Il fattore di equità viene fissato a 1.5, rappresentando un fattore di tolleranza rispetto alla quota esattamente equa dal momento in cui la ripartizione della banda disponibile di TCP non è mai perfettamente uniforme.

### Logica a stati

Una volta stabilita la soglia sopra la quale identifichiamo un traffico come anomalo, ai fini di mitigare l'attacco è stata implementata una logica a stati per evitare che picchi momentanei di traffico possano causare un intervento non necessario

Due stati possibili:

- NORMALE, il flusso rispetta la propria quota.
- BLOCCATO, il flusso non rispetta la propria quota per due intervalli consecutivi con condizione di contesa, in particolare la durata del blocco cresce ad ogni recidiva (60, 120, 300 secondi).

## Mitigazione

All'interno del controller, la condizione di dichiarazione di un **flusso** come malevolo è la seguente:

```
if over and contention:
    fs.block_count += 1
else:
    fs.block_count = 0

if fs.block_count >= CONFIRM_BLOCK:
    reason = "sopra la quota equa sotto contesa"
    if self._apply_block(client, reason):
        self._transition(client, BLOCCATO, reason)
```

Al verificarsi della precedente condizione, la regola che viene predisposta all'interno del relativo switch è la seguente e viene configurata con un livello di priorità pari a 100, superiore alla priorità delle altre rotte di inoltro. In particolare le due regole coesistono ma OpenFlow applicherà quella a priorità maggiore.

```
self.add_flow(datapath, PRIO_BLOCK,
              self._block_match(datapath, client), [])

def _block_match(self, datapath, client):
    parser = datapath.ofproto_parser
    return parser.OFPMatch(eth_type=ether_types.ETH_TYPE_IP,
                           ipv4_src=client,
                           ipv4_dst=SERVER)
```

Dal momento in cui il match comprende l'indirizzo IP sorgente, il flusso relativo agli altri host legittimi collegati allo stesso switch non è interessato

## Scenari applicativi

Sono stati realizzati due scenari che differiscono solo per il rate generato dall'host h2. In entrambi i casi h1 e h3 generano traffico TCP a 4 Mbps verso il server, mentre h2 genera traffico UDP variabile a seconda dello scenario

### Scenario legittimo

All'interno di questo scenario, l'host H2 genera un traffico regolare a 3Mbps.

Comandi mininet per la configurazione dei porti del server:

```
h4 iperf -s -p 5001 & (inizializzazione della porta TCP del server H4)
```

```
h4 iperf -s -u -p 5002 & (inizializzazione della porta UDP del server H4)
```

Comandi per la generazione di traffico:

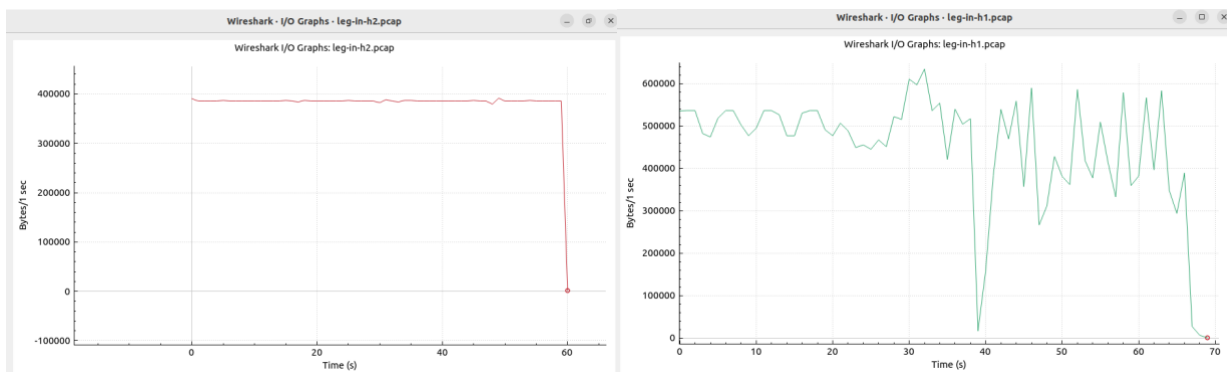
h1 iperf -c 10.0.0.4 -p 5001 -b 4m -t 60 & (generazione del traffico di H1 tramite iperf)

h2 iperf -c 10.0.0.4 -u -p 5002 -b 3m -t 60 & (generazione del traffico di H2 tramite iperf)

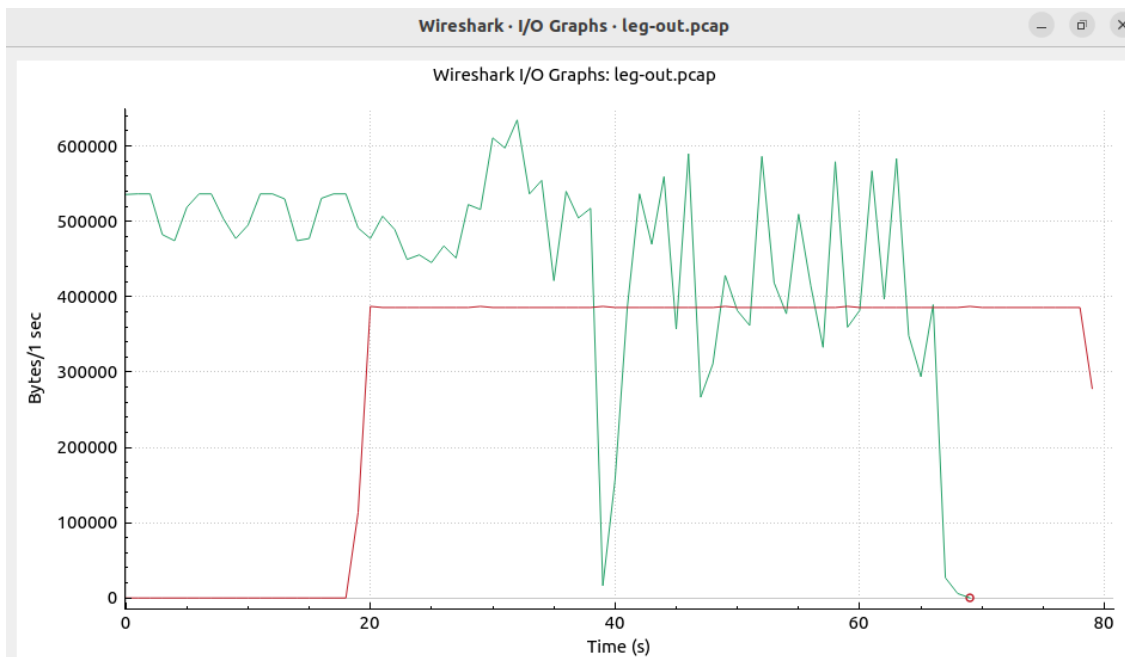
h3 iperf -c 10.0.0.4 -p 5001 -b 4m -t 60 & (generazione del traffico di H3 tramite iperf)

Nelle seguenti immagini vengono mostrati gli andamenti dei flussi del client legittimo e del client attaccante in ingresso ed in uscita dallo switch S1. Come si può notare, in questo caso, il traffico entrante è esattamente uguale al traffico uscente dal momento in cui l'attaccante sta generando del traffico sotto la soglia massima consentita nel caso di 3 host attivi.

Traffico in ingresso sullo switch S1:



Traffico in uscita sullo switch S1:



## Scenario con attacco DoS

All'interno di questo scenario, l'host H2 genera un traffico malevolo a 8Mbps.

Comandi per la generazione di traffico:

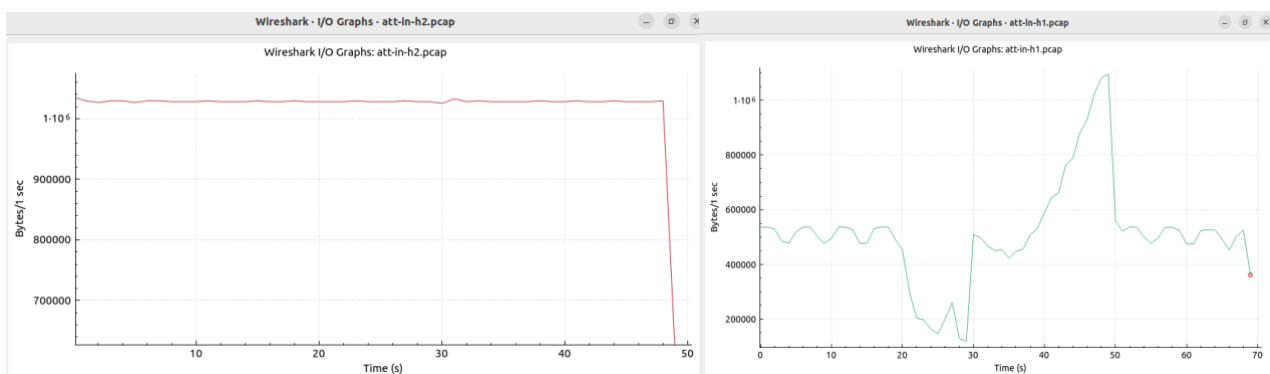
```
h1 iperf -c 10.0.0.4 -p 5001 -b 4m -t 90 &
```

```
h3 iperf -c 10.0.0.4 -p 5001 -b 4m -t 90 &
```

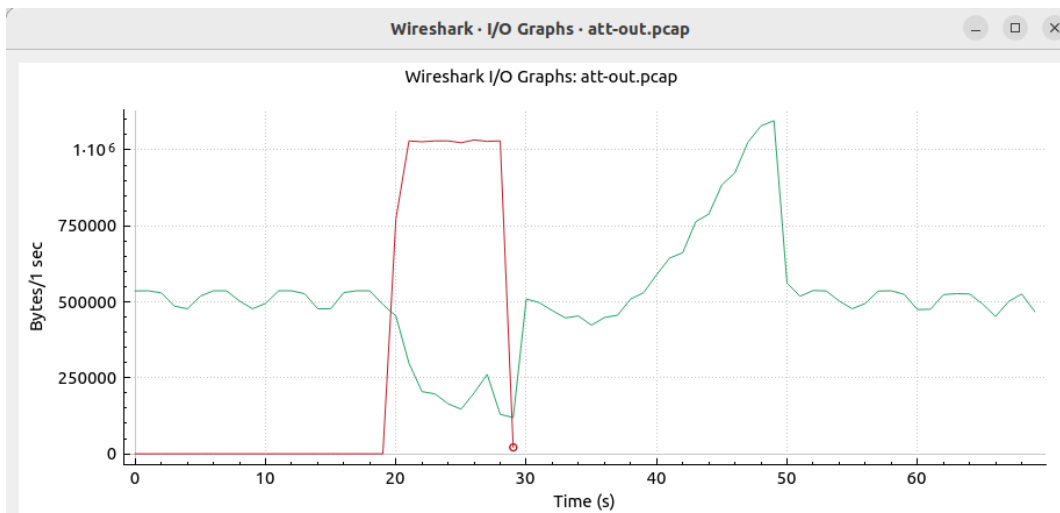
```
h2 iperf -c 10.0.0.4 -u -p 5002 -b 8m -t 60 &
```

In questo caso, come anticipato, il traffico generato dall'attaccante supera la soglia di equità per un periodo di tempo maggiore a quello consentito tale che sulla porta di uscita dello switch S1, dopo circa 10 secondi dall'immissione del traffico illegittimo, il traffico malevolo viene azzerato, mostrando la successiva stabilizzazione del rate del flusso sano.

Traffico in ingresso sullo switch S1:



Traffico in uscita sullo switch S1:



Candidati:

Moccia Paolo DE9000195

D'Angelo Pierluigi DE9000130

Ferraro Salvatore DE9000176